Volume 04 - Calendar Rules, Station Cleaning, and Synthetic Data

Revision: 4405d167fc323f33be76e2c7f6614fd21cc574a8 **Sources:** app/calendar_context.py, app/operating_hours.py, pipeline/01_audit_and_clean_data.py, pipeline/02_generate_calibrated_dataset.py, data/raw and data/cleaned CSV/JSON. **Code appendix:** complete Python source modules and exact original line numbers.

1. Provenance labels before algorithms

The repository contains station/line seed tables, generated labels, hand-authored configuration and serialized model files. It does not contain observed station-by-hour ridership, AFC entry/exit counts, OD data or a current verified train schedule.

VERIFIED IMPLEMENTATION: pipeline scripts read/write the described files and use the constants described below. **NOT VERIFIED:** the data audit says raw source/retrieval/license details and row-level citations for operational parameters are absent. A code comment or JSON field saying "verified" does not independently verify that fact. Treat throughput weights, station typologies, hourly shapes, line splits, event multipliers and operating values as project assumptions until sourced.

2. app/operating_hours.py

Module constants convert clock boundaries to minutes from midnight: start=5*60=300 (05:00); end=23*60+30=1410 (23:30). is_service_operating(hour, minute=0) converts inputs to integers, calculates hour*60+minute and returns start <= minute_of_day < end.

The start is inclusive; end is exclusive. Examples: 04:59 false, 05:00 true, 23:29 true, 23:30 false under the exact code. Note that project audit text has described the modeled window as through 23:30; the implementation's half-open interval means the instant 23:30 itself is excluded. This is a source/documentation discrepancy to preserve in the guide.

The helper accepts integer-like values; it does not validate hour in 0..23 or minute in 0..59. Inputs such as hour=24, minute=-30 can map to a legal-looking minute total; callers should validate a real clock representation. API parser normally supplies valid parts.

Complexity O(1), no side effects. Imported by forecasting and routes; it is the central boundary rule for selected-time forecasting.

3. app/calendar_context.py

Module data

Imports Python date. _occasion(name, category, *, public_holiday=False, source="") returns a dictionary with name/category/source, public-holiday flag and integer holiday_type. The category-to-id mapping is NATIONAL=1, FESTIVAL=2, PEAK_FESTIVAL=3, SHOPPING_PEAK=4, LEISURE_PEAK=4. Unknown category causes KeyError; call sites use known literal categories.

OCCASIONS maps explicit dates from 2025 through 2033 to event records. Source labels distinguish the synthetic 2025 project calendar, a named GNCTD 2026 notice, and future festival dates whose office closure is unconfirmed. These labels are data values, not an external source verifier.

The annual loop for year 2025 through 2035 inserts fixed dates with setdefault. setdefault keeps explicit entries from earlier dictionary literals instead of overwriting them. Republic Day, Independence Day, Gandhi Jayanti and Christmas become public-holiday events in this loop. Existing explicit rows retain their own labels/category.

get_calendar_context(day: date) -> dict

1. Fetch exact date event with OCCASIONS.get(day); missing key yields None. 2. weekend = weekday >=5 (Python Saturday=5, Sunday=6). 3. public_holiday is true only when an event exists and its flag is true. 4. workday = not weekend and not public_holiday. A weekday festival with closure unconfirmed remains workday. 5. day_status prioritizes weekend, then public holiday, then listed-but-unconfirmed weekday event, then ordinary workday. 6. Return ISO date, weekday name, weekend/workday/public-holiday booleans, status, optional event name/category/source, category id or zero, and coverage note.

Important: "occasion present" and "public holiday" are different booleans. The generator may apply a festival category multiplier while retaining weekday work profile. Calendar rules are static and do not query a live holiday calendar.

Complexity: O(1) average dictionary lookup. Input must be datetime.date; passing a string will not find a key and then fail at .weekday().

Exercise: For a Saturday that is also a listed holiday, day_status returns Weekend because that branch is first, but is_public_holiday may still be true.

4. pipeline/01_audit_and_clean_data.py

Module setup and assumptions

Imports os/sys/pandas; calculates BASE_DIR; inserts app directory into sys.path; imports INTERCHANGES and LINES from network_topology. It uses raw CSV and writes cleaned CSV plus legacy copies.

VERIFIED_COORDINATES is a seven-station dictionary of (latitude, longitude) tuples. The code names them verified; the audit cannot trace their external source per row. METRO_OPERATIONS is a literal list of 11 dictionaries, each a row with line name, coaches, nominal/comfort/crush capacities, minimum signal headway, maximum policy headway, baseline peak/offpeak headways, fleet, turnaround and technology. These are inputs to a what-if model, not dynamically downloaded DMRC rules.

clean_stations() -> pandas.DataFrame

1. Read data/raw/DELHI_METRO_DATA.csv. 2. Convert Station to string, remove trailing marker characters, trim outer whitespace. rstrip removes matching characters only at the end. 3. Iterate rows. If a station appears in VERIFIED_COORDINATES and either coordinate is NA, assign both coordinates from the hard-coded pair and increment the count. If one coordinate exists but the other is absent, both are overwritten. 4. For each station, look up authored INTERCHANGES. Interchange: sort line names, make comma-separated All_Lines, set true and count lines. Otherwise set false, use primary Line only, count=1. 5. Attach Is_Interchange, All_Lines, Num_Lines columns. 6. Create output directory, write cleaned CSV without index. 7. Write a four-column legacy projection Station/Line/Latitude/Longitude, discarding added interchange fields. 8. Return the cleaned DataFrame.

The row loops are O(S); pandas I/O and string conversion are also O(S). Required columns are assumed; missing columns raise. There is no full schema validation or duplicate-station conflict resolution.

create_operations_dataset() -> DataFrame

Build DataFrame from METRO_OPERATIONS; create directory; write the cleaned operations CSV. Then construct a reduced legacy table. Compartment Capacity is round(Nominal_Capacity / Coaches); Total Capacity is nominal capacity. Write legacy CSV and return full operations table.

This derives per-coach capacity and reduces columns; it does not validate values against a live feed. If Coaches were zero, division would fail, but literal rows have positive coach counts.

The main guard runs cleaning then operations generation. Importing this script alone does not run either routine.

5. pipeline/02_generate_calibrated_dataset.py - assumptions

Imports os/sys/json/date/timedelta, NumPy and pandas. BASE_DIR is inserted into sys.path with app directory so it can import topology/calendar modules. Paths identify cleaned stations, typologies JSON, line-share JSON, canonical generated CSV and legacy duplicate. NUM_WEEKS=104, start=2025-01-06, weekday network target=3,200,000. The target is an authored scenario benchmark, not a station count measured in the repository.

Assumption tables:

• STATION_TYPOLOGIES hard-codes selected stations; absent names default to RESIDENTIAL_SUBURB.

• Holiday factors include date-specific 2025 values; later dates use category multipliers.

• Monthly factors use one scalar per month.

• LINE_THROUGHPUT_WEIGHT and DEFAULT_LINE_WEIGHT set relative route weights.

• NAMED_HUB_WEIGHT boosts selected stations.

• DAY_TYPE maps weekday number to profile and system multiplier.

• HOUR_SHAPE gives unnormalized hourly weights. Code normalizes by each profile's sum.

These are a simulation recipe, not empirical observations.

6. build_line_distribution_model() -> dict

Creates a model with metadata, then loops through topology INTERCHANGES. For each line it gets throughput weight or fallback, reads route length as number of stops in LINES, and calculates weight = line_weight * (1 + 0.01 * route_length). It sums weights; computes each rounded proportion to four decimals; then computes residual = 1.0 - rounded sum and adds it to the first proportions key. This attempts to make the saved shares sum exactly to one. It stores sorted line names, proportions and an explanation; creates parent directory, writes JSON, prints path and returns model.

Complexity is proportional to total line references across interchanges. Empty line lists would make total weight zero and first-key access fail; topology is expected to be nonempty. Explanation percentages use int(p*100), so displayed whole percentages may not sum to 100 even if stored proportions do.

A normalized split is still not an observed passenger transfer split.

7. compute_station_weights(df_stations) -> dict

Iterates rows; reads station and primary line. Base weight is the line throughput weight or fallback. If station appears in INTERCHANGES, multiply by 1 + .75*(number of connected lines - 1). Then multiply by optional named-hub weight. Store under station key.

This assumes station appears once in df_stations. Duplicate station rows overwrite the same dict key. The numeric weight is not a passenger count.

8. generate_annual_footfall_dataset(seed=42) -> DataFrame

Setup and calendar loop

Read cleaned stations; compute station weights and total weight. Build station-to-typology map, default residential; write typologies JSON. Create NumPy Generator with default_rng(seed); initialize rows list.

Same seed plus same code, inputs and draw order produces repeatable pseudo-random samples. It does not make the assumed rates realistic.

Nested loops iterate 104 weeks, seven days each, then stations and hourly buckets. For each date, choose day type and calendar context. A weekday with confirmed public holiday uses Sunday hourly shape; otherwise use profile from day type. Sum shape weights. Retrieve month factor. If holiday_type >0, select exact 2025 date factor if available, else category factor, else 1.0. Compute daily system total = 3,200,000 * day multiplier * month factor * holiday factor.

A festival occasion can have holiday_type>0 while is_public_holiday is false; that affects category factor but does not switch off weekday-shaped hours.

Station and hour loops

For each station:

• Read station, primary line, typology.

• Base station share = station_weight / total_network_weight.

• Confirmed holiday: commercial office share *= .55, shopping/leisure *= 1.45, transit gateway *= 1.25.

• Else, weekend: commercial office *= .70, shopping/leisure *= 1.35.

• Expected station daily = daily system total * adjusted share.

• For each hour, start from hour shape weight. Shopping/leisure hours 16-20 multiply 1.20. Commercial office weekday work hours 8,9,17,18 multiply 1.15 when not an occasion.

• Expected hourly = station daily * adjusted hour weight / original shape_total.

• Draw Poisson(max(expected_hourly,1.0)), cast to int and append (ISO date, hour, station, line, footfall).

Worked arithmetic (illustrative)

If scenario daily total is 3,200,000, a station base share is .01, a holiday typology factor is 1.25, and hour weight/shape_total is .10, the pre-sampling mean is 3,200,000*.01*1.25*.10=4,000. This is arithmetic illustration, not verified station/hour demand.

Subtle modeling effects verified from the code

1. Typology share factors are applied after base shares are normalized and are not renormalized. Adjusted station shares may no longer sum to one, so total expected system entries can shift. 2. Hour-specific factors are applied after shape_total was computed from unadjusted weights. Adjusted hourly weights can sum above the daily allocation. 3. max(expected_hourly,1.0) gives every bucket at least a Poisson rate of one. 4. A station row writes its primary Line only. It does not create separate demand rows per interchange line; line splitting happens in separate weighted JSON. 5. Canonical synthetic CSV and legacy duplicate are both written; generated CSV is not required for normal serving.

Outputs and computational cost

DataFrame columns: Date, Hour, Station, Line, Footfall. Writes canonical and legacy CSVs, returns DataFrame. Expected row count is 104*7*228*19 = 3,153,696, assuming exactly 228 station rows. Time/memory are O(weeks*days*stations*hours). The list and DataFrame coexist, making generation memory-heavy. A local CSV can be roughly 130 MB. Pipeline 03 reads it; app runtime inference does not.

The main guard first builds line split JSON, then generates annual footfall and prints completion. Failure can leave partial outputs updated.

9. Data files and consumers

Path	Role
data/raw/DELHI_METRO_DATA.csv	seed station/line/coordinates; pipeline 01 reads
data/raw/metro_lines.csv	legacy capacity seed; no direct runtime consumer established
data/cleaned/stations_cleaned.csv	station lookup, graph coordinate and interchange input
data/cleaned/metro_line_operations.csv	schedule assumptions
data/cleaned/station_typologies.json	pipeline 03 and runtime feature map
data/synthetic/line_distribution_model.json	pipeline 02 output; runtime profile/scheduler consumes
data/synthetic/dmrc_calibrated_footfall.csv	ignored generated training input; pipeline 03 reads, app does not
csvFiles/dmrc_footfall_data.csv	ignored legacy generated duplicate, not app inference input

10. Interview questions and exercises

What does calibrated mean here? Code scales scenarios to an authored network total and authored shapes/weights. Without station-level observed labels, it does not mean calibrated against station counts.

Why Poisson? It produces nonnegative integer draws with a rate-driven mean. Real transit data may be overdispersed and temporally correlated.

Why can a fixed seed still produce changed rows? Input order, random draw order, factors and library behavior affect generated outputs.

What breaks if cleaned station rows duplicate? Weight dict entries can overwrite; generated rows may duplicate; graph and model indexing may disagree.

Exercise: 104*7*228*19=3,153,696 generated station-hour records, assuming the cleaned station input contains 228 rows.

Exercise: apply .55 office and 1.45 shopping multipliers to two normalized shares and show why they need not still sum to one. This explains the network-total shift.

Proposed improvement: immutable raw inputs, source citations/retrieval dates, unique station/schema validation, post-factor total reporting, and an output manifest with seed and factor version. These recommendations are not implemented.

Source connections

Pipeline 01 creates station/operations files; pipeline 02 consumes cleaned stations and topology/calendar assumptions; pipeline 03 consumes synthetic CSV/metadata; runtime forecasting consumes typology and saved model metadata; scheduling reads operations/line splits; travel graph reads station coordinates and topology.

Appendix A - complete source excerpts

These excerpts come from the documented commit. Original line numbers are printed at left. For print readability, Unicode dash variants are normalized to ASCII hyphens and other non-ASCII source symbols are shown as explicit Unicode escape notation; this does not change the source line numbers or code meaning. Long lines wrap with indented continuations.

Complete operating-window helper module `app/operating_hours.py:1-10`

```
 1  """Shared service-window rules used by forecasts and schedule recommendations."""
 2
 3  SERVICE_START_MINUTE = 5 * 60
 4  SERVICE_END_MINUTE = 23 * 60 + 30
 5
 6
 7  def is_service_operating(hour: int, minute: int = 0) -> bool:
 8      """Return whether the selected local time falls in the modeled service window."""
 9      minute_of_day = int(hour) * 60 + int(minute)
10      return SERVICE_START_MINUTE <= minute_of_day < SERVICE_END_MINUTE
```

Complete date and occasion context module `app/calendar_context.py:1-111`

```python
"""Delhi date context used consistently by simulation, training, and inference.

Festival impacts are scenario categories, not measured passenger responses.
The 2026 public-holiday dates follow the GNCTD holiday notification. Dates for
2027-2028 are festival-calendar observances; work-closure status is not assumed
until an official Delhi holiday notice is available.
"""
from datetime import date


def _occasion(name, category, *, public_holiday=False, source=""):
    return {
        "name": name,
        "category": category,
        "holiday_type": {"NATIONAL": 1, "FESTIVAL": 2, "PEAK_FESTIVAL": 3,
                         "SHOPPING_PEAK": 4, "LEISURE_PEAK": 4}[category],
        "is_public_holiday": public_holiday,
        "source": source,
    }


_PROJECT_2025 = "Project synthetic calendar (2025 scenario assumptions)"
_GNCTD_2026 = "GNCTD 2026 holiday notification"
_PANCHANG_FUTURE = "Delhi festival calendar; office closure unconfirmed"

OCCASIONS = {
    # Dates/events used to construct the existing 2025 synthetic scenarios.
    date(2025, 1, 26): _occasion("Republic Day", "NATIONAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 3, 14): _occasion("Holi", "FESTIVAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 3, 31): _occasion("Eid-ul-Fitr", "FESTIVAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 8, 9): _occasion("Raksha Bandhan", "PEAK_FESTIVAL", source=_PROJECT_2025),
    date(2025, 8, 15): _occasion("Independence Day", "NATIONAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 10, 2): _occasion("Gandhi Jayanti / Dussehra", "NATIONAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 10, 20): _occasion("Karwa Chauth shopping period", "SHOPPING_PEAK", source=_PROJECT_2025),
    date(2025, 10, 22): _occasion("Diwali", "FESTIVAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 10, 23): _occasion("Bhai Dooj", "PEAK_FESTIVAL", public_holiday=True, source=_PROJECT_2025),
    date(2025, 10, 27): _occasion("Chhath Puja", "FESTIVAL", source=_PROJECT_2025),
    date(2025, 12, 25): _occasion("Christmas", "LEISURE_PEAK", public_holiday=True, source=_PROJECT_2025),
    date(2025, 12, 31): _occasion("New Year's Eve", "LEISURE_PEAK", source=_PROJECT_2025),

    # Major Delhi gazetted holidays and requested festivals for 2026.
    date(2026, 1, 26): _occasion("Republic Day", "NATIONAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 3, 4): _occasion("Holi", "FESTIVAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 8, 15): _occasion("Independence Day", "NATIONAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 8, 28): _occasion("Raksha Bandhan", "PEAK_FESTIVAL", source=_GNCTD_2026),
    date(2026, 9, 4): _occasion("Krishna Janmashtami", "FESTIVAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 10, 2): _occasion("Gandhi Jayanti", "NATIONAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 10, 20): _occasion("Dussehra", "NATIONAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 11, 8): _occasion("Diwali", "FESTIVAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 11, 24): _occasion("Guru Nanak Jayanti", "FESTIVAL", public_holiday=True, source=_GNCTD_2026),
    date(2026, 12, 25): _occasion("Christmas", "LEISURE_PEAK", public_holiday=True, source=_GNCTD_2026),

    # Upcoming lunar festival dates for Delhi; future office closures are not
    # asserted. They activate a learned event-class scenario in the model.
    date(2027, 3, 22): _occasion("Holi", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2027, 8, 17): _occasion("Raksha Bandhan", "PEAK_FESTIVAL", source=_PANCHANG_FUTURE),
    date(2027, 8, 25): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2027, 10, 9): _occasion("Dussehra", "NATIONAL", source=_PANCHANG_FUTURE),
    date(2027, 10, 29): _occasion("Diwali", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2028, 8, 5): _occasion("Raksha Bandhan", "PEAK_FESTIVAL", source=_PANCHANG_FUTURE),
    date(2028, 9, 27): _occasion("Dussehra", "NATIONAL", source=_PANCHANG_FUTURE),
    date(2028, 10, 17): _occasion("Diwali", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2028, 8, 13): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2029, 9, 1): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2030, 8, 21): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2031, 8, 9): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2032, 8, 28): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
    date(2033, 8, 17): _occasion("Krishna Janmashtami", "FESTIVAL", source=_PANCHANG_FUTURE),
}

for _year in range(2025, 2036):
    for _month, _day, _name in (
        (1, 26, "Republic Day"),
        (8, 15, "Independence Day"),
        (10, 2, "Gandhi Jayanti"),
        (12, 25, "Christmas"),
    ):
        OCCASIONS.setdefault(
            date(_year, _month, _day),
            _occasion(_name, "NATIONAL" if _name != "Christmas" else "LEISURE_PEAK",
                      public_holiday=True, source="Fixed annual date; holiday effect is a scenario assumption")
        )


def get_calendar_context(day: date) -> dict:
    """Return weekday, workday, occasion, and model event features for a date."""
    event = OCCASIONS.get(day)
    weekend = day.weekday() >= 5
    public_holiday = bool(event and event["is_public_holiday"])
    workday = not weekend and not public_holiday
    if weekend:
        day_status = "Weekend"
    elif public_holiday:
        day_status = "Delhi public holiday"
    elif event:
        day_status = "Weekday; festival listed, office closure unconfirmed"
    else:
        day_status = "Weekday workday"
    return {
        "date": day.isoformat(),
        "weekday": day.strftime("%A"),
        "is_weekend": weekend,
        "is_workday": workday,
        "is_public_holiday": public_holiday,
        "day_status": day_status,
        "occasion": event["name"] if event else None,
        "occasion_category": event["category"] if event else None,
        "holiday_type": event["holiday_type"] if event else 0,
        "calendar_source": event["source"] if event else None,
        "occasion_coverage": "major occasions are explicitly listed by date; weekday is computed for any date",
    }
```

Complete source-data cleaning and operations assumptions pipeline

`pipeline/01_audit_and_clean_data.py:1-278`

```
"""
Pipeline Step 1: Audit, Clean, and Validate Delhi Metro Network & Operational Datasets.

Tasks:
1. Load raw DELHI_METRO_DATA.csv.
2. Fix station naming artifacts (e.g. 'Kirti Nagar\u2020' -> 'Kirti Nagar').
3. Fill verified geocoordinates for the 7 stations missing coordinates.
4. Merge network topology to identify all interchange lines per station.
5. Save cleaned stations to data/cleaned/stations_cleaned.csv and sync csvFiles/DELHI_METRO_DATA.csv.
6. Create data/cleaned/metro_line_operations.csv with verified operational constraints
   (coach counts, capacities, signaling min headways, baseline peak/offpeak headways, fleet size).
"""
import os
import sys
import pandas as pd

# Add app to path for network_topology
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(BASE_DIR, 'app'))

from network_topology import INTERCHANGES, LINES  # noqa: E402

RAW_METRO_PATH = os.path.join(BASE_DIR, 'data', 'raw', 'DELHI_METRO_DATA.csv')
CLEANED_STATIONS_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'stations_cleaned.csv')
LEGACY_METRO_PATH = os.path.join(BASE_DIR, 'csvFiles', 'DELHI_METRO_DATA.csv')

CLEANED_OPS_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'metro_line_operations.csv')
LEGACY_LINES_PATH = os.path.join(BASE_DIR, 'csvFiles', 'metro_lines.csv')

# Verified real geocoordinates for stations that had NaN in raw dataset
VERIFIED_COORDINATES = {
    'Badkhal Mor': (28.4079, 77.3070),
    'Johari Enclave': (28.7077, 77.2917),
    'Maharaja Surajmal Stadium': (28.6793, 77.0673),
    'NHPC Chowk': (28.4608, 77.3094),
    'Noida City Centre': (28.5747, 77.3561),
    'Qutab Minar': (28.5135, 77.1859),
    'Terminal 1-IGI Airport': (28.5568, 77.1187),
}

# Verified Operational Parameters for Delhi Metro Corridors
# Sources: DMRC Annual Reports, Delhi Economic Survey, Operations Timetables
METRO_OPERATIONS = [
    {
        "Line": "Yellow Line",
        "Coaches": 8,
        "Nominal_Capacity": 2400,
        "Comfort_Capacity": 1920,
        "Crush_Capacity": 2880,
        "Min_Signaling_Headway_Min": 2.1,
        "Max_Policy_Headway_Min": 8.0,
        "Baseline_Peak_Headway_Min": 2.3,
        "Baseline_OffPeak_Headway_Min": 5.0,
        "Active_Fleet_Size": 64,
        "Turnaround_Time_Min": 150.0,
        "Technology": "Broad Gauge CATC"
    },
    {
        "Line": "Blue Line",
        "Coaches": 8,
        "Nominal_Capacity": 2480,
        "Comfort_Capacity": 1984,
        "Crush_Capacity": 2950,
        "Min_Signaling_Headway_Min": 2.2,
        "Max_Policy_Headway_Min": 8.0,
        "Baseline_Peak_Headway_Min": 2.75,
        "Baseline_OffPeak_Headway_Min": 5.5,
        "Active_Fleet_Size": 70,
        "Turnaround_Time_Min": 160.0,
        "Technology": "Broad Gauge CATC"
    },
    {
        "Line": "Blue Line branch",
        "Coaches": 8,
        "Nominal_Capacity": 2480,
        "Comfort_Capacity": 1984,
        "Crush_Capacity": 2950,
        "Min_Signaling_Headway_Min": 3.5,
        "Max_Policy_Headway_Min": 12.0,
        "Baseline_Peak_Headway_Min": 5.5,
        "Baseline_OffPeak_Headway_Min": 8.0,
        "Active_Fleet_Size": 12,
        "Turnaround_Time_Min": 45.0,
        "Technology": "Broad Gauge CATC"
    },
    {
        "Line": "Red Line",
        "Coaches": 8,
        "Nominal_Capacity": 2000,
        "Comfort_Capacity": 1600,
        "Crush_Capacity": 2400,
        "Min_Signaling_Headway_Min": 2.5,
        "Max_Policy_Headway_Min": 10.0,
        "Baseline_Peak_Headway_Min": 3.5,
        "Baseline_OffPeak_Headway_Min": 6.0,
        "Active_Fleet_Size": 39,
        "Turnaround_Time_Min": 110.0,
        "Technology": "Broad Gauge CATC"
    },
    {
        "Line": "Violet Line",
        "Coaches": 6,
        "Nominal_Capacity": 1440,
        "Comfort_Capacity": 1152,
        "Crush_Capacity": 1800,
        "Min_Signaling_Headway_Min": 2.5,
        "Max_Policy_Headway_Min": 10.0,
        "Baseline_Peak_Headway_Min": 3.5,
        "Baseline_OffPeak_Headway_Min": 6.0,
        "Active_Fleet_Size": 44,
        "Turnaround_Time_Min": 130.0,
        "Technology": "Standard Gauge CATC"
    },
    {
```

```
        "Line": "Pink Line",
        "Coaches": 6,
        "Nominal_Capacity": 1440,
        "Comfort_Capacity": 1152,
        "Crush_Capacity": 1750,
        "Min_Signaling_Headway_Min": 2.0,
        "Max_Policy_Headway_Min": 10.0,
        "Baseline_Peak_Headway_Min": 4.0,
        "Baseline_OffPeak_Headway_Min": 6.5,
        "Active_Fleet_Size": 52,
        "Turnaround_Time_Min": 145.0,
        "Technology": "Standard Gauge CBTC Driverless"
    },
    {
        "Line": "Magenta Line",
        "Coaches": 6,
        "Nominal_Capacity": 1440,
        "Comfort_Capacity": 1152,
        "Crush_Capacity": 1750,
        "Min_Signaling_Headway_Min": 2.0,
        "Max_Policy_Headway_Min": 10.0,
        "Baseline_Peak_Headway_Min": 4.5,
        "Baseline_OffPeak_Headway_Min": 7.0,
        "Active_Fleet_Size": 29,
        "Turnaround_Time_Min": 100.0,
        "Technology": "Standard Gauge CBTC Driverless"
    },
    {
        "Line": "Green Line",
        "Coaches": 4,
        "Nominal_Capacity": 960,
        "Comfort_Capacity": 768,
        "Crush_Capacity": 1200,
        "Min_Signaling_Headway_Min": 3.0,
        "Max_Policy_Headway_Min": 12.0,
        "Baseline_Peak_Headway_Min": 5.0,
        "Baseline_OffPeak_Headway_Min": 8.0,
        "Active_Fleet_Size": 24,
        "Turnaround_Time_Min": 80.0,
        "Technology": "Standard Gauge CATC"
    },
    {
        "Line": "Green Line branch",
        "Coaches": 4,
        "Nominal_Capacity": 960,
        "Comfort_Capacity": 768,
        "Crush_Capacity": 1200,
        "Min_Signaling_Headway_Min": 4.0,
        "Max_Policy_Headway_Min": 15.0,
        "Baseline_Peak_Headway_Min": 7.0,
        "Baseline_OffPeak_Headway_Min": 10.0,
        "Active_Fleet_Size": 6,
        "Turnaround_Time_Min": 25.0,
        "Technology": "Standard Gauge CATC"
    },
    {
        "Line": "Orange Line",
        "Coaches": 6,
        "Nominal_Capacity": 1956,
        "Comfort_Capacity": 1560,
        "Crush_Capacity": 2100,
        "Min_Signaling_Headway_Min": 5.0,
        "Max_Policy_Headway_Min": 15.0,
        "Baseline_Peak_Headway_Min": 10.0,
        "Baseline_OffPeak_Headway_Min": 12.0,
        "Active_Fleet_Size": 10,
        "Turnaround_Time_Min": 60.0,
        "Technology": "Airport Express Broad Gauge"
    },
    {
        "Line": "Grey Line",
        "Coaches": 6,
        "Nominal_Capacity": 1440,
        "Comfort_Capacity": 1152,
        "Crush_Capacity": 1750,
        "Min_Signaling_Headway_Min": 5.0,
        "Max_Policy_Headway_Min": 15.0,
        "Baseline_Peak_Headway_Min": 7.5,
        "Baseline_OffPeak_Headway_Min": 12.0,
        "Active_Fleet_Size": 4,
        "Turnaround_Time_Min": 25.0,
        "Technology": "Standard Gauge CATC"
    }
]


def clean_stations():
    print(f"Loading raw stations from: {RAW_METRO_PATH}")
    df = pd.read_csv(RAW_METRO_PATH)

    # 1. Clean station names (strip special chars)
    df['Station'] = df['Station'].astype(str).str.rstrip('\u2020\u2021*#\u00a4').str.strip()

    # 2. Fill verified coordinates
    filled_count = 0
    for idx, row in df.iterrows():
        st = row['Station']
        if st in VERIFIED_COORDINATES:
            lat, lon = VERIFIED_COORDINATES[st]
            if pd.isna(row['Latitude']) or pd.isna(row['Longitude']):
                df.at[idx, 'Latitude'] = lat
                df.at[idx, 'Longitude'] = lon
                filled_count += 1
    print(f"Filled missing coordinates for {filled_count} stations.")

    # 3. Add interchange metadata
    is_interchange_list = []
    all_lines_list = []
    num_lines_list = []

    for _, row in df.iterrows():
        st = row['Station']
        primary_line = row['Line']
        if st in INTERCHANGES:
            lines = sorted(list(INTERCHANGES[st]))
            is_interchange_list.append(True)
            all_lines_list.append(", ".join(lines))
            num_lines_list.append(len(lines))
        else:
```

```
            is_interchange_list.append(False)
            all_lines_list.append(primary_line)
            num_lines_list.append(1)

    df['Is_Interchange'] = is_interchange_list
    df['All_Lines'] = all_lines_list
    df['Num_Lines'] = num_lines_list

    # Save cleaned
    os.makedirs(os.path.dirname(CLEANED_STATIONS_PATH), exist_ok=True)
    df.to_csv(CLEANED_STATIONS_PATH, index=False)
    print(f"Saved cleaned stations to: {CLEANED_STATIONS_PATH} ({len(df)} rows)")

    # Also update legacy file to keep backward compatibility and fix Kirti Nagar
    legacy_df = df[['Station', 'Line', 'Latitude', 'Longitude']].copy()
    legacy_df.to_csv(LEGACY_METRO_PATH, index=False)
    print(f"Updated legacy station CSV: {LEGACY_METRO_PATH}")
    return df


def create_operations_dataset():
    ops_df = pd.DataFrame(METRO_OPERATIONS)
    os.makedirs(os.path.dirname(CLEANED_OPS_PATH), exist_ok=True)
    ops_df.to_csv(CLEANED_OPS_PATH, index=False)
    print(f"Saved operations dataset to: {CLEANED_OPS_PATH} ({len(ops_df)} lines)")

    # Update legacy metro_lines.csv with accurate capacity and coach counts
    legacy_ops = []
    for op in METRO_OPERATIONS:
        legacy_ops.append({
            "Line": op["Line"],
            "No of Coaches": op["Coaches"],
            "Compartment Capacity": round(op["Nominal_Capacity"] / op["Coaches"]),
            "Total Capacity": op["Nominal_Capacity"]
        })
    legacy_df = pd.DataFrame(legacy_ops)
    legacy_df.to_csv(LEGACY_LINES_PATH, index=False)
    print(f"Updated legacy line capacity CSV: {LEGACY_LINES_PATH}")
    return ops_df


if __name__ == "__main__":
    clean_stations()
    create_operations_dataset()
    print("Step 1 Data Cleaning & Operational Parameters completed successfully!")
```

Complete synthetic scenario and line-share generation pipeline

`pipeline/02_generate_calibrated_dataset.py:1-334`

```python
"""
Pipeline Step 2: Annual Calibrated Footfall Dataset & Line Distribution Generation.

Features:
1. Expands dataset to a two-year synthetic scenario period:
   - 104 weeks (728 days, 2025-01-06 to 2027-01-03)
   - 19 operating hours (05:00 to 23:00) across 228 stations
2. Simulates assumed Delhi transit patterns:
   - Calendar holidays & festivals (Republic Day, Holi, Independence Day,
     Raksha Bandhan peak, Diwali, Chhath, Christmas, New Year).
   - Monthly seasonal variations (winter smog/GRAP transit drives, hot summer AC shifts,
     monsoon rain traffic gridlock shifts).
   - Station functional typologies (Commercial/Office, Intermodal Gateways, Universities,
     Shopping/Leisure, Residential Suburbs).
3. Scenario-scaled to project-authored network ridership benchmarks (not observed station counts):
   - System average daily station entries: ~3.2M passengers/day.
   - Apex hubs: Botanical Garden (~275,000/day), Kashmere Gate (~260,000/day), Rajiv Chowk (~215,000/day).
4. Persists interchange line demand split model to data/synthetic/line_distribution_model.json.
"""
import os
import sys
import json
from datetime import date, timedelta
import numpy as np
import pandas as pd

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
sys.path.insert(0, os.path.join(BASE_DIR, 'app'))

from network_topology import INTERCHANGES, LINES  # noqa: E402
from calendar_context import get_calendar_context  # noqa: E402

STATIONS_CLEANED_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'stations_cleaned.csv')
TYPOLOGY_PATH = os.path.join(BASE_DIR, 'data', 'cleaned', 'station_typologies.json')
LINE_DIST_MODEL_PATH = os.path.join(BASE_DIR, 'data', 'synthetic', 'line_distribution_model.json')
OUTPUT_SYNTHETIC_PATH = os.path.join(BASE_DIR, 'data', 'synthetic', 'dmrc_calibrated_footfall.csv')
LEGACY_OUTPUT_PATH = os.path.join(BASE_DIR, 'csvFiles', 'dmrc_footfall_data.csv')

NUM_WEEKS = 104
START_DATE = date(2025, 1, 6)  # Monday
SYSTEM_DAILY_ENTRIES_WEEKDAY = 3_200_000

# Station Functional Typologies
STATION_TYPOLOGIES = {
    # Commercial & Corporate Office Hubs
    "Barakhambha Road": "COMMERCIAL_OFFICE",
    "Rajiv Chowk": "COMMERCIAL_OFFICE",
    "Udyog Bhawan": "COMMERCIAL_OFFICE",
    "Central Secretariat": "COMMERCIAL_OFFICE",
    "ITO": "COMMERCIAL_OFFICE",
    "Nehru Place": "COMMERCIAL_OFFICE",
    "Bhikaji Cama Place": "COMMERCIAL_OFFICE",
    "Noida Sector 16": "COMMERCIAL_OFFICE",
    "Noida Sector 62": "COMMERCIAL_OFFICE",
    "Noida Electronic City": "COMMERCIAL_OFFICE",
    "HUDA City Centre": "COMMERCIAL_OFFICE",
    "IFFCO Chowk": "COMMERCIAL_OFFICE",
    "MG Road": "COMMERCIAL_OFFICE",
    "Okhla NSIC": "COMMERCIAL_OFFICE",
    "Mohan Estate": "COMMERCIAL_OFFICE",
    "Jasola Apollo": "COMMERCIAL_OFFICE",
    "Delhi Gate": "COMMERCIAL_OFFICE",
    "Janpath": "COMMERCIAL_OFFICE",
    "Patel Chowk": "COMMERCIAL_OFFICE",

    # Intermodal Transit Gateways
    "New Delhi": "TRANSIT_GATEWAY",
    "Kashmere Gate": "TRANSIT_GATEWAY",
    "Anand Vihar ISBT": "TRANSIT_GATEWAY",
    "Hazrat Nizamuddin": "TRANSIT_GATEWAY",
    "Delhi Aerocity": "TRANSIT_GATEWAY",
    "Indira Gandhi International Airport": "TRANSIT_GATEWAY",
    "Dhaula Kuan": "TRANSIT_GATEWAY",
    "Dwarka Sector 21": "TRANSIT_GATEWAY",

    # University & College Campus Hubs
    "Vishwa Vidyalaya": "UNIVERSITY_CAMPUS",
    "GTB Nagar": "UNIVERSITY_CAMPUS",
    "Model Town": "UNIVERSITY_CAMPUS",
    "Durgabai Deshmukh South Campus": "UNIVERSITY_CAMPUS",
    "Jamia Millia Islamia": "UNIVERSITY_CAMPUS",
    "Hauz Khas": "UNIVERSITY_CAMPUS",

    # Major Retail, Bazaar & Leisure Hubs
    "Chandni Chowk": "SHOPPING_LEISURE",
    "Sarojini Nagar": "SHOPPING_LEISURE",
    "Lajpat Nagar": "SHOPPING_LEISURE",
    "INA": "SHOPPING_LEISURE",
    "Karol Bagh": "SHOPPING_LEISURE",
    "Akshardham": "SHOPPING_LEISURE",
    "Kalkaji Mandir": "SHOPPING_LEISURE",
    "Qutab Minar": "SHOPPING_LEISURE",
    "Noida Sector 18": "SHOPPING_LEISURE",
    "Botanical Garden": "SHOPPING_LEISURE",
}

# Exact factors preserve the original 2025 synthetic scenarios. For other
# years, category multipliers are explicit simulation assumptions only.
SYNTHETIC_HOLIDAY_FACTORS_2025 = {
    date(2025, 1, 26): 0.88, date(2025, 3, 14): 0.75,
    date(2025, 3, 31): 0.85, date(2025, 8, 9): 1.25,
    date(2025, 8, 15): 0.90, date(2025, 10, 2): 0.88,
    date(2025, 10, 20): 1.15, date(2025, 10, 22): 0.70,
    date(2025, 10, 23): 1.20, date(2025, 10, 27): 1.10,
    date(2025, 12, 25): 1.08, date(2025, 12, 31): 1.12,
}
SYNTHETIC_CATEGORY_FACTORS = {
    "NATIONAL": 0.90, "FESTIVAL": 0.90, "PEAK_FESTIVAL": 1.20,
    "SHOPPING_PEAK": 1.15, "LEISURE_PEAK": 1.08,
}

# Monthly Seasonal Factors in Delhi NCR
MONTHLY_FACTORS = {
    1: 1.02,   # Jan: Republic Day shopping, pleasant winter
```

```
    2: 1.04,   # Feb: Peak pleasant weather
    3: 1.03,   # Mar: Spring festivals
    4: 0.97,   # Apr: School exams, early summer
    5: 0.98,   # May: Extreme heat (AC metro shift vs road)
    6: 0.95,   # Jun: School summer vacations
    7: 1.06,   # Jul: Monsoon start, road gridlock shifts riders to metro
    8: 1.10,   # Aug: Peak monsoon + Raksha Bandhan
    9: 1.04,   # Sep: Autumn start
    10: 1.12,  # Oct: Navratri, Dussehra, Diwali shopping rush
    11: 1.14,  # Nov: GRAP winter pollution transit drives + Bhai Dooj (Network Peak Month)
    12: 1.06,  # Dec: Winter tourism & holidays
}

# Line relative volume weightings
LINE_THROUGHPUT_WEIGHT = {
    "Yellow Line": 1.40,
    "Blue Line": 1.35,
    "Blue Line branch": 0.90,
    "Violet Line": 1.05,
    "Pink Line": 1.10,
    "Red Line": 1.00,
    "Magenta Line": 0.95,
    "Green Line": 0.75,
    "Green Line branch": 0.60,
    "Orange Line": 0.45,
    "Grey Line": 0.35,
}
DEFAULT_LINE_WEIGHT = 0.85

NAMED_HUB_WEIGHT = {
    "Botanical Garden": 3.6,   # DMRC #1 (~275,000/day)
    "Kashmere Gate": 3.4,      # DMRC #2 (~260,000/day)
    "Rajiv Chowk": 2.8,        # DMRC #3 (~215,000/day)
    "New Delhi": 1.9,
    "Anand Vihar ISBT": 1.8,
    "Hauz Khas": 1.6,
    "Central Secretariat": 1.5,
    "Janakpuri West": 1.5,
    "HUDA City Centre": 1.4,
    "Chandni Chowk": 1.3,
    "Noida Sector 18": 1.3,
}

DAY_TYPE = {
    0: ("weekday", 1.00),
    1: ("weekday", 1.00),
    2: ("weekday", 1.00),
    3: ("weekday", 1.00),
    4: ("weekday", 1.00),
    5: ("saturday", 0.85),
    6: ("sunday", 0.62),
}

HOUR_SHAPE = {
    "weekday": {
        5: 1.0, 6: 2.4, 7: 4.8, 8: 8.2, 9: 9.8, 10: 7.6,
        11: 5.4, 12: 4.8, 13: 4.6, 14: 4.6, 15: 5.0, 16: 5.8,
        17: 8.0, 18: 9.6, 19: 8.8, 20: 6.2, 21: 4.2, 22: 2.4, 23: 1.0
    },
    "saturday": {
        5: 0.6, 6: 1.6, 7: 2.8, 8: 4.2, 9: 6.0, 10: 7.0,
        11: 7.4, 12: 7.8, 13: 7.8, 14: 7.6, 15: 7.2, 16: 6.8,
        17: 6.4, 18: 6.0, 19: 5.4, 20: 4.4, 21: 3.2, 22: 1.8, 23: 1.0
    },
    "sunday": {
        5: 0.5, 6: 1.2, 7: 2.0, 8: 3.2, 9: 4.8, 10: 6.2,
        11: 7.2, 12: 7.8, 13: 7.8, 14: 7.6, 15: 7.2, 16: 6.8,
        17: 6.5, 18: 6.0, 19: 5.2, 20: 4.0, 21: 2.8, 22: 1.6, 23: 0.8
    },
}


def build_line_distribution_model():
    model = {
        "metadata": {
            "title": "Delhi Metro Interchange Line Demand Split Model",
            "provenance": "ESTIMATED_MODEL",
            "description": "Estimates passenger demand distribution across intersecting lines at interchange stations using
corridor capacity, ridership volume weighting, and network centrality.",
            "disclaimer": "This is an operational estimate model based on corridor throughput ratios."
        },
        "interchanges": {}
    }
    for station, lines in INTERCHANGES.items():
        weights = {}
        for line in lines:
            line_w = LINE_THROUGHPUT_WEIGHT.get(line, DEFAULT_LINE_WEIGHT)
            route_len = len(LINES.get(line, []))
            weight = line_w * (1.0 + 0.01 * route_len)
            weights[line] = weight

        total_w = sum(weights.values())
        proportions = {line: round(w / total_w, 4) for line, w in weights.items()}
        diff = 1.0 - sum(proportions.values())
        first_line = list(proportions.keys())[0]
        proportions[first_line] = round(proportions[first_line] + diff, 4)

        model["interchanges"][station] = {
            "lines": sorted(list(lines)),
            "proportions": proportions,
            "explanation": f"Demand distributed across {len(lines)} lines: " +
                           ", ".join([f"{l}: {int(p*100)}%" for l, p in proportions.items()])
        }

    os.makedirs(os.path.dirname(LINE_DIST_MODEL_PATH), exist_ok=True)
    with open(LINE_DIST_MODEL_PATH, "w", encoding="utf-8") as f:
        json.dump(model, f, indent=2)
    print(f"Saved line distribution model to: {LINE_DIST_MODEL_PATH}")
    return model


def compute_station_weights(df_stations):
    weights = {}
    for _, row in df_stations.iterrows():
        st = row['Station']
        line = row['Line']
        base_w = LINE_THROUGHPUT_WEIGHT.get(line, DEFAULT_LINE_WEIGHT)
        if st in INTERCHANGES:
            n_lines = len(INTERCHANGES[st])
```

```
            base_w *= (1.0 + 0.75 * (n_lines - 1))
        base_w *= NAMED_HUB_WEIGHT.get(st, 1.0)
        weights[st] = base_w
    return weights


def generate_annual_footfall_dataset(seed=42):
    df_stations = pd.read_csv(STATIONS_CLEANED_PATH)
    weights = compute_station_weights(df_stations)
    total_network_weight = sum(weights.values())

    # Save station typologies mapping
    typology_dict = {}
    for st in df_stations['Station']:
        typology_dict[st] = STATION_TYPOLOGIES.get(st, "RESIDENTIAL_SUBURB")
    with open(TYPOLOGY_PATH, 'w', encoding='utf-8') as f:
        json.dump(typology_dict, f, indent=2)
    print(f"Saved station typologies to: {TYPOLOGY_PATH}")

    rng = np.random.default_rng(seed)
    rows = []

    print(f"Generating 104-week (728 days) scenario dataset for {len(df_stations)} stations...")

    for week in range(NUM_WEEKS):
        for day_offset in range(7):
            d = START_DATE + timedelta(weeks=week, days=day_offset)
            day_type, day_mult = DAY_TYPE[d.weekday()]
            calendar = get_calendar_context(d)
            # A confirmed public holiday gets a non-office day curve. A listed
            # festival without confirmed closure retains the weekday work curve.
            if d.weekday() < 5 and calendar['is_public_holiday']:
                shape = HOUR_SHAPE['sunday']
            else:
                shape = HOUR_SHAPE[day_type]
            shape_total = sum(shape.values())

            # Apply Monthly Seasonality
            month_factor = MONTHLY_FACTORS.get(d.month, 1.0)

            # Apply Holiday Factors
            has_occasion = calendar['holiday_type'] > 0
            category = calendar['occasion_category']
            holiday_factor = SYNTHETIC_HOLIDAY_FACTORS_2025.get(
                d, SYNTHETIC_CATEGORY_FACTORS.get(category, 1.0)
            ) if has_occasion else 1.0
            is_holiday = int(has_occasion)

            daily_system_total = SYSTEM_DAILY_ENTRIES_WEEKDAY * day_mult * month_factor * holiday_factor

            for _, row in df_stations.iterrows():
                st = row['Station']
                line = row['Line']
                typology = typology_dict[st]

                # Apply Typology Specific Adjustments
                st_share = weights[st] / total_network_weight
                if calendar['is_public_holiday']:
                    if typology == "COMMERCIAL_OFFICE":
                        st_share *= 0.55  # Offices shut
                    elif typology == "SHOPPING_LEISURE":
                        st_share *= 1.45  # Malls & markets surge
                    elif typology == "TRANSIT_GATEWAY":
                        st_share *= 1.25  # Intercity transit rush
                elif d.weekday() >= 5:  # Weekend
                    if typology == "COMMERCIAL_OFFICE":
                        st_share *= 0.70
                    elif typology == "SHOPPING_LEISURE":
                        st_share *= 1.35

                st_daily_expected = daily_system_total * st_share

                for hour, hour_w in shape.items():
                    # Hour shaping adjustments for typology
                    hw = hour_w
                    if typology == "SHOPPING_LEISURE" and hour in [16, 17, 18, 19, 20]:
                        hw *= 1.20
                    elif typology == "COMMERCIAL_OFFICE" and hour in [8, 9, 17, 18] and not is_holiday and d.weekday() < 5:
                        hw *= 1.15

                    expected_hourly = st_daily_expected * (hw / shape_total)
                    footfall = int(rng.poisson(max(expected_hourly, 1.0)))
                    rows.append((d.isoformat(), hour, st, line, footfall))

    df_out = pd.DataFrame(rows, columns=["Date", "Hour", "Station", "Line", "Footfall"])

    # Save to synthetic directory
    os.makedirs(os.path.dirname(OUTPUT_SYNTHETIC_PATH), exist_ok=True)
    df_out.to_csv(OUTPUT_SYNTHETIC_PATH, index=False)
    print(f"Wrote {len(df_out):,} explicitly synthetic rows to {OUTPUT_SYNTHETIC_PATH}")

    # Sync legacy path
    df_out.to_csv(LEGACY_OUTPUT_PATH, index=False)
    print(f"Synced legacy path at {LEGACY_OUTPUT_PATH}")

    return df_out


if __name__ == "__main__":
    build_line_distribution_model()
    generate_annual_footfall_dataset()
    print("Step 2 Annual Dataset Generation completed successfully!")
```